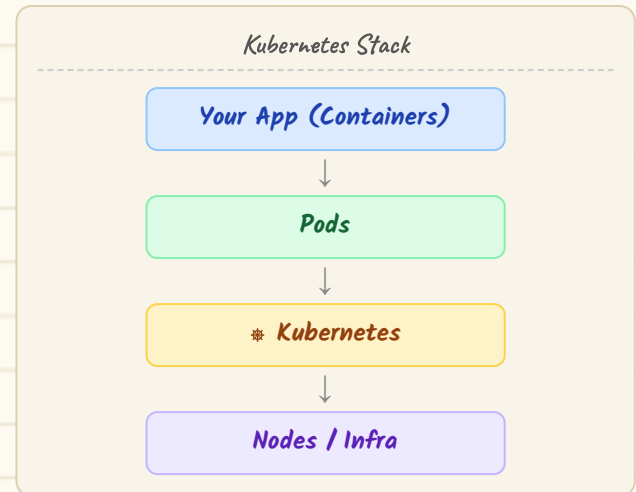


What is Kubernetes?

1 What is Kubernetes?

- **Kubernetes** is an open-source **container orchestration** platform.
- Originally developed by **Google**, now maintained by CNCF.
- Automates **deployment, scaling** and management of containerized apps.
- Also known as **K8s** (K + 8 letters + s).
- Works with Docker, containerd, CRI-O runtimes.



2 Why Kubernetes?



Auto Scaling

Scale up/down based on load automatically



Self Healing

Restarts crashed containers automatically



Rolling Updates

Zero-downtime deployments



Load Balancing

Distributes traffic across pods



Bin Packing

Optimally places containers on nodes



Secrets Mgmt

Manages sensitive config data

★ Remember

- ✓ Kubernetes does **NOT** run your code -- it orchestrates containers
- ✓ K8s = open-source, cloud-native, declarative configuration



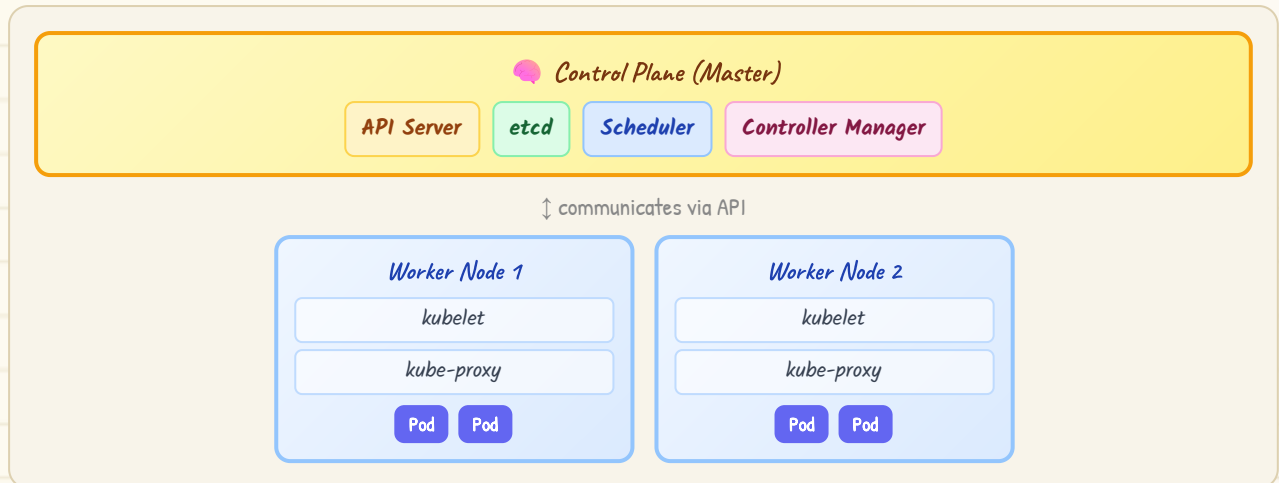
Interview Tip

Q: What is Kubernetes?

A: Kubernetes is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications.

Kubernetes Architecture

1 Cluster Overview



2 Control Plane vs Worker Nodes

Control Plane	Worker Node
<code>kube-apiserver</code> -- central gateway	<code>kubelet</code> -- node agent
<code>etcd</code> -- key-value store	<code>kube-proxy</code> -- network rules
<code>kube-scheduler</code> -- assigns pods	Container Runtime (Docker/CRI-O)
<code>controller-manager</code> -- maintains state	Pods run here

★ Key Points

- ✓ `etcd` stores **all** cluster state
- ✓ API Server is the only entry point
- ✓ Scheduler only assigns, doesn't run pods

💡 Interview Tip

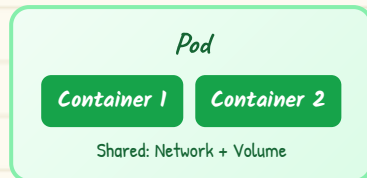
Q: What is `etcd`?

A: `etcd` is a distributed key-value store that stores the entire cluster state and configuration.

Pods & Deployments

1 What is a Pod?

- **Smallest deployable** unit in Kubernetes
- Contains **one or more** containers
- Containers share **network & storage**
- Has a unique IP address inside the cluster
- Ephemeral -- pods can die and be replaced

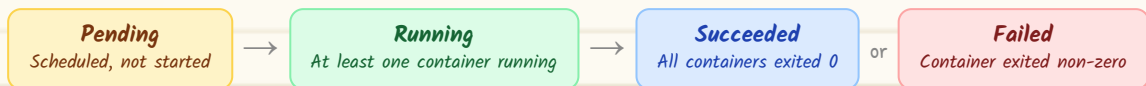


2 What is a Deployment?

- Manages **ReplicaSets** of pods
- Declares **desired state** (e.g., "3 replicas")
- Handles **rolling updates** & rollbacks
- Self-heals -- replaces failed pods automatically



3 Pod Lifecycle



Interview Tip

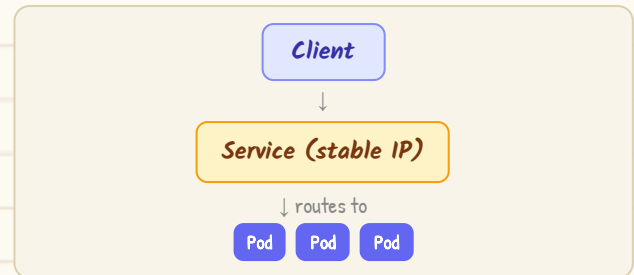
Q: Difference between Pod and Deployment?

A: A Pod is a single instance of your app. A Deployment manages a group of identical pods with features like scaling, rolling updates, and self-healing.

Services & Networking

1 What is a Service?

- A stable network endpoint to **expose pods**
- Pods have dynamic IPs -- Services give a **stable IP**
- Acts as a **load balancer** across pod replicas
- Uses **label selectors** to find target pods



2 Types of Services

Type	Access	Use Case
ClusterIP	Only within cluster	Internal microservice comms
NodePort	External via Node IP:Port	Dev/Testing exposure
LoadBalancer	External via Cloud LB	Production traffic (AWS/GCP)
ExternalName	DNS CNAME to external	Access external DB etc.

3 Ingress

- HTTP/HTTPS routing to services
- Supports **host-based** routing
- Supports **path-based** routing
- Requires an **Ingress Controller** (nginx, traefik)



Interview Tip

Q: LoadBalancer vs Ingress?

A: LoadBalancer per-service (expensive). Ingress is one entry point routing to multiple services -- better for HTTP.

Config, Secrets & Volumes

1 ConfigMap

- Stores **non-sensitive** configuration data
- Key-value pairs or entire config files
- Injected as env vars or mounted as volumes
- Example: DB host, app port, feature flags

```
apiVersion: v1
kind: ConfigMap
data:
  DB_HOST: "postgres"
  APP_PORT: "8080"
```

2 Secrets

- Stores **sensitive** data (passwords, tokens)
- Base64 encoded (**NOT encrypted by default!**)
- Should enable etcd encryption at rest
- Types: Opaque, TLS, docker-registry

```
apiVersion: v1
kind: Secret
type: Opaque
data:
  password: cGFzc3dvcmQ=
```

3 Volumes & Persistent Storage

emptyDir

Temp storage, deleted when pod dies

hostPath

Mounts node filesystem path

PersistentVolume

Cluster-wide storage resource (PV)

PVC

Pod's claim to use a PV

★ ConfigMap vs Secret

- ✓ ConfigMap = non-sensitive config, plain text
- ✓ Secret = sensitive data, base64 encoded (enable encryption at rest!)

Use Cases & Quick Reference

1 Real-World Use Cases

Microservices

Run 100s of microservices independently, scale each separately

CI/CD Pipelines

Jenkins, ArgoCD, Tekton run on K8s for build and deploy

ML Workloads

GPU workloads, model training and serving at scale

Multi-Cloud

Portable workloads across AWS, GCP, Azure

Data Platforms

Kafka, Spark, Flink run on Kubernetes clusters

E-Commerce

Netflix, Airbnb, Spotify scale on demand

2 kubectl Quick Reference

```
kubectl get pods
```

List all pods

```
kubectl describe pod <name>
```

Detailed pod info

```
kubectl logs <pod>
```

View pod logs

```
kubectl apply -f file.yaml
```

Deploy from YAML

```
kubectl delete pod <name>
```

Delete a pod

```
kubectl scale deploy/app --replicas=5
```

Scale deployment

★ K8s Cheat Sheet

- ✓ Pod -- smallest unit
- ✓ Deployment -- manages ReplicaSets
- ✓ Service -- stable network endpoint
- ✓ Ingress -- HTTP routing gateway
- ✓ ConfigMap -- non-sensitive config
- ✓ Secret -- sensitive data

💡 Final Tip

Practice: Install minikube locally and deploy a simple app. Hands-on experience beats 10x reading!

```
minikube start
```